

Relatório de verificação do projeto NetLab G1

Relatório de verificação do projeto NetLab G1 - web scraping

Repositório: netlab-g1

URL remota: <https://github.com/fregulha/netlab-g1-webscraping.git>

Branch: main...origin/main

Commit: 04ce8ff

Data do relatório: 17/09/2026 16:41:04

Resumo executivo:

- O repositório contém a rotina de extração em Python com BeautifulSoup, testes automatizados, dados de amostra, métricas e documentação.
- A implementação cobre boa parte dos requisitos solicitados, com destaque para robustez de parsing, deduplicação, logs, CSV/JSON e validação automatizada.
- Há limitações relevantes em paginação online e em coleta em tempo real, pois a página G1 é renderizada dinamicamente e a validação de navegação interativa não foi concluída nesta sessão.

Arquivos principais:

- scraper.py: rotina de coleta, parsing, deduplicação, exportação, diagnósticos
- evaluate.py: métricas de completude, cobertura e qualidade
- tests/test_scraper.py: testes automatizados
- README.md: documentação de diagnóstico e execução
- data/coleta_multilotes/resultados.json: base coletada em múltiplos lotes
- data/metricas_multilotes.json: métricas produzidas
- docs/llm.md: proposta de uso de LLM

Verificação de requisitos:

1. Identificar problemas do código atual: STATUS = OK (README documenta seletores antigos, paginação e ausência de deduplicação).
2. Corrigir rotina com Python + BeautifulSoup: STATUS = OK (scraper.py usa BeautifulSoup e parsing dos cards do G1).
3. Atualizar seletores e paginação: STATUS = PARCIAL (seletores ajustados para widget--info / video-widget--info; paginação online ainda não validada em ambiente real).
4. Coletar título, URL, resumo, data, página e horário: STATUS = OK (campos presentes na estrutura de dados e exportação).
5. Tratar falhas de conexão / HTTP / campos ausentes: STATUS = OK (tratamento de exceptions, timeout, Content-Type, campos nulos e quarentena).
6. Evitar duplicados: STATUS = OK (deduplicação por URL com preservação da primeira ocorrência).
7. Organização e reutilização: STATUS = OK (funções modulares e separadas por responsabilidade).
8. Logs e diagnósticos: STATUS = OK (logging com eventos e aviso de zero cards).
9. Salvar em CSV/JSON: STATUS = OK (resultados.csv e resultados.json).
10. Testes automatizados: STATUS = OK (24 testes aprovados em 0.34s).
11. Documentação de requisitos e execução: STATUS = OK (README.md com instruções e diagnóstico).
12. Proposta de uso de LLM: STATUS = OK (docs/llm.md).
13. Avaliação de qualidade dos dados: STATUS = OK (README + evaluate.py + métricas de referência).
14. Limitações e melhorias: STATUS = OK (README descreve pendências).

Evidência de testes:

Comando executado: `python -m pytest -q`

Resultado: 24 passed in 0.34s

Dados coletados e métricas:

- 30 URLs únicas
- 26 notícias e 4 vídeos
- 30 registros validados no lote cumulativo mais recente
- 30 duplicatas removidas no processamento de lotes cumulativos
- Título, URL, resumo e data exibida presentes em 30/30
- data_publicacao permanece nula porque a página exibe data de atualização, não de publicação

Conclusão:

O projeto atende aos requisitos da maior parte da proposta e está em um estado consistente para uso acadêmico e avaliação seletiva. O principal ponto pendente é a validação de paginação online em ambiente real e a confirmação de coleta dinâmica do portal em produção.